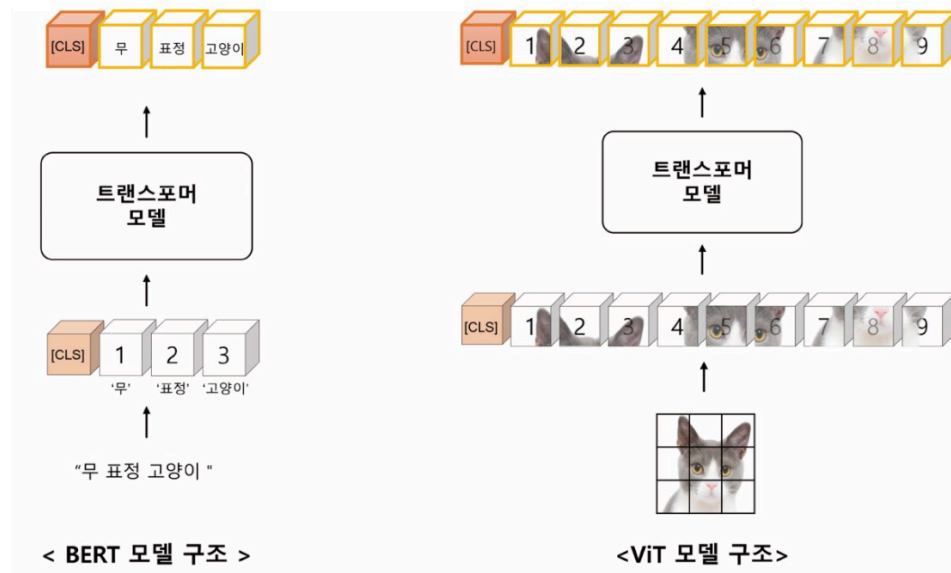


[자연어처리와 컴퓨터비전 심층학습] 10장

ViT(Vision Transformer)

- 배경
 - 이미지를 자연어 처리 방식처럼 분류해보려는 시도
 - 트랜스포머 모델에서 사용되는 셀프 어텐션 적용(이미지 분류 모델에서 사용하는 CNN 모델 합성곱 계층 방법이 아님)
 - = 이미지 분류를 위한 지역 특징 추출이 아닌, 전체 이미지를 한 번에 처리하는 방식
 -
- 정리
 - 트랜스포머 구조 자체를 컴퓨터비전 분야에 적용한 첫 번째 연구
 - BERT 모델과 ViT 모델의 차이
 - 입력 데이터를 만든 과정에 차이가 있음



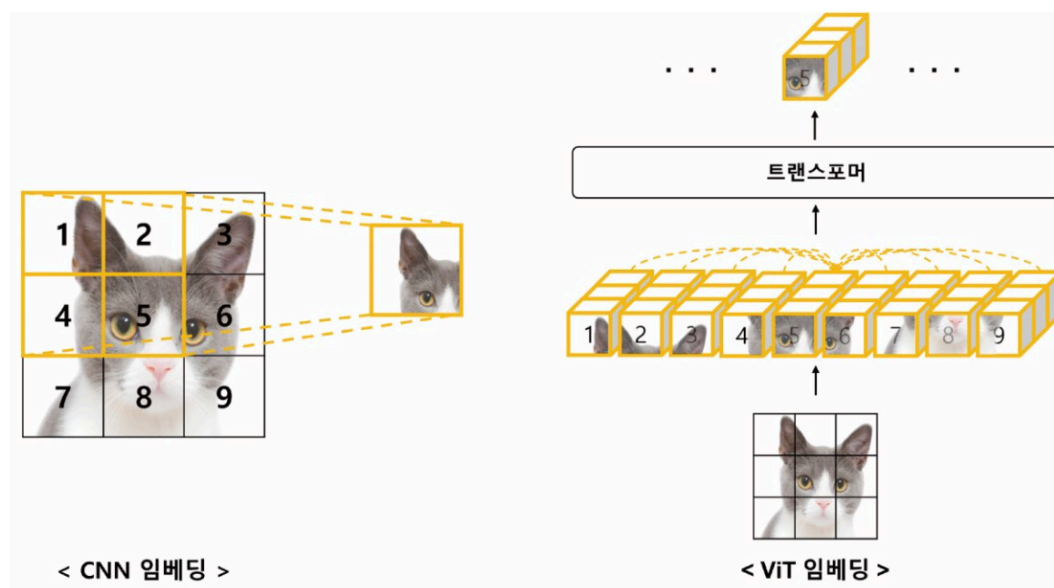
- BERT: 문장보다 작은 단위의 토큰들이 순차적으로 입력
- ViT: 이미지 격자로 작은 단위의 이미지 패치가 나뉘어 순차적으로 입력

(왼쪽에서 오른쪽, 위에서 아래로 표현된 시퀀셜 배열 가정)

합성곱 모델과 ViT 모델 비교

두 모델 모두 이미지 특징을 잘 표현하는 임베딩을 만들고자 하는 목적을 가짐. 다만 그 과정에서 차이가 있음

- 합성곱
 - 이미지 패치 중 일부만 선택하여 학습
 - 이를 통해 이미지 전체의 특징 추출
- ViT 임베딩
 - 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계 학습
 - 이를 위해 셀프 어텐션 방법을 사용해 모든 이미지 패치가 서로에게 주는 영향을 고려해 이미지의 전체 특징 추출
 - 따라서 모든 이미지 패치가 학습에 관여하며 높은 수준의 이미지 표현을 제공할 수 있음



합성곱 신경망은 고양이의 오른쪽 눈을 표현하는 데 1,2,4,5만 학습됨
반면 ViT 임베딩은 셀프 어텐션 방법으로 모든 이미지 패치가 학습에 관여함

- ViT 임베딩의 장점
 - 전체 이미지 정보를 표현하기 위해 수많은 계층이 필요한 합성곱 신경망과 달리 어텐션 거리를 계산해 오직 한 개의 ViT 레이어로 전체 이미지 정보를 쉽게 표현할 수 있음

- 픽셀 단위로 처리하는 합성곱 모델과 달리 패치 단위로 이미지를 처리해 더 작은 모델로도 높은 성능을 얻을 수 있음
- ViT 임베딩의 취약점
 - 입력 이미지 크기가 고정되어 있어 크기가 다른 이미지 처리를 위해서는 전처리 과정을 거쳐야 함
 - ViT는 공간적인 위치 정보를 고려하지 않고 패치 간 상대적인 위치 정보만 고려하기 때문에 이미지 변환에 취약함

ViT의 귀납적 편향

- 귀납적 편향
 - 의미: 일반화 성능 향상을 위한 모델의 가정
(ex. 이미지 데이터는 공간적 관계를 잘 표현하므로 지역적 편향을 가진 합성곱 신경망 모델이 많이 사용됨)
 - 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

- 합성곱 신경망은 이미지를 작은 조각으로 나누어 각 조각의 특징을 추출한 후 조합해 전체 이미지의 특징 추론
→ 지역적인 특징을 강조해 이미지 데이터에서 좋은 성능 발휘
- 순환 신경망은 시퀀셜 편향을 가져, 시간적 관계를 잘 표현하는 시계열 데이터를 다룰 때 사용됨
- ViT 모델은 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 일반화된 관계를 학습해 귀납적 편향이 거의 없음
- 귀납적 편향의 가정이 옳바르다면, 모델은 더 적은 데이터로 높은 일반화 성능을 보임. 하지만 너무 강한 귀납적 편향은 다른 유형의 데이터나 관계를 표현하는 데 어려움을 초래할 수 있음

ViT 모델

입력 이미지를 트랜스포커 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환하는 패치 임베딩과

각 패치와의 관계를 학습하는 인코더 계층으로 구성됨

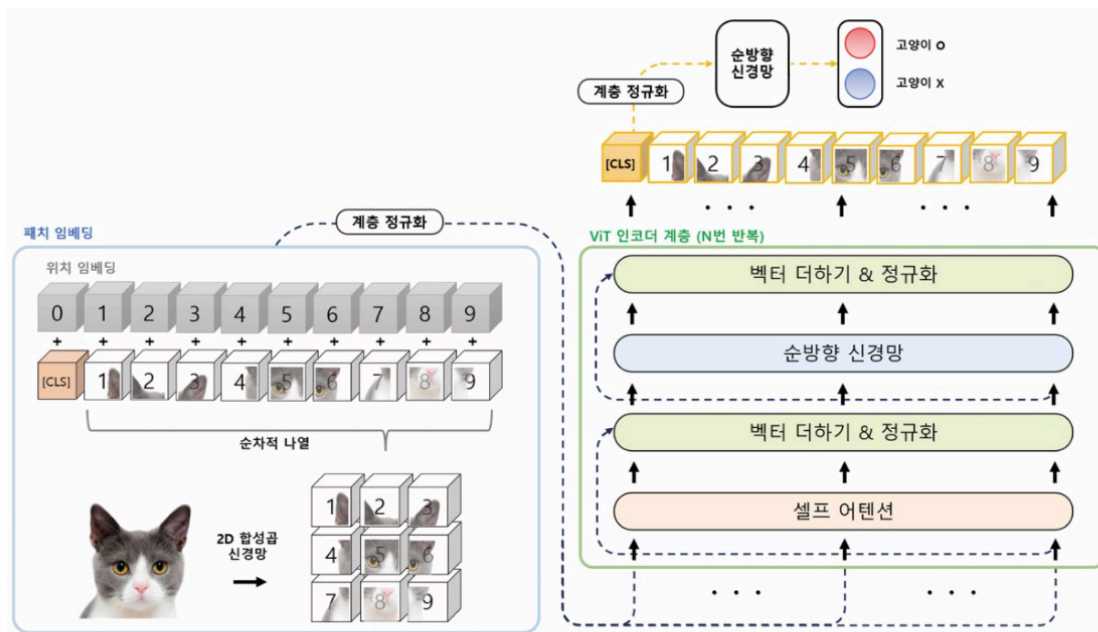


그림 10.4 ViT 모델 구조

패치 임베딩

- 입력 이미지를 작은 패치로 분할하는 과정
- 패치 임베딩에 앞서, 이미지 크기를 맞추는 전처리가 수행되어야 함
→ 전체 이미지를 패치 크기로 분할해 시퀀셜 배열을 만들(합성곱 신경망 계층 활용)

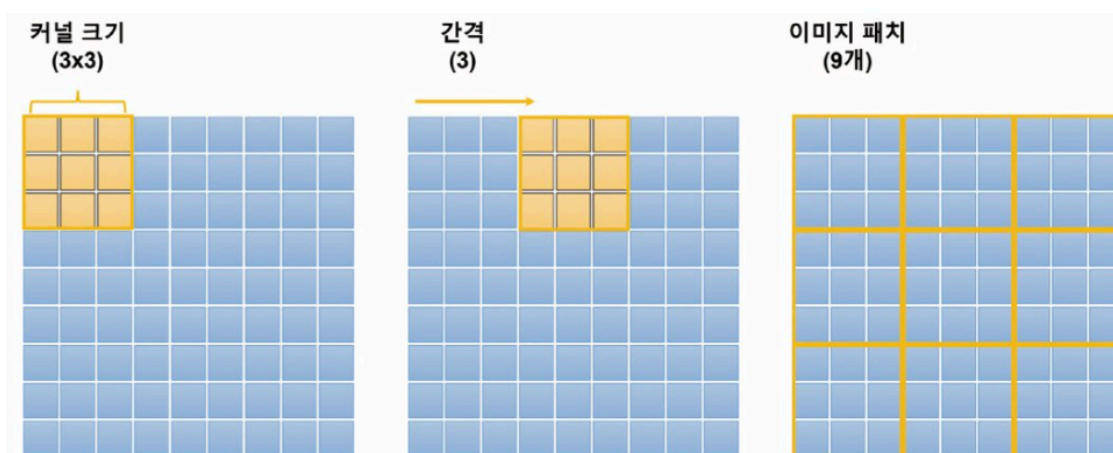


그림 10.5 이미지 패치 생성 과정

→ 커널 크기(=패치의 크기)와 간격(=패치가 이동하는 폭) 설정(하이퍼파라미터에 정의)

→ 분할된 이미지 배치들을 배열 형태로 왼-오, 위-아래 순으로 배열

→ 배열 가장 왼쪽에 **분류 토큰** 추가(전체 이미지를 대표하는 벡터로 특정 문제를 예측하는 데 사용됨)

→ 위치 임베딩을 사용해 인접 패치 간의 관계 학습(패치의 위치 정보를 임베딩 벡터로 변환해 기존 패치 벡터와 더함)

→ 계층 정규화 적용

→ 패칭 임베딩 만들어짐!

이렇게 입력 이미지를 작은 패치로 나눠 처리해 GPU 메모리의 한계를 극복하고 더 큰 이미지를 처리할 수 있음

인코더 계층

- 각 패치를 벡터로 변환한 후, 인코더 계층에 입력으로 전달해 다양한 쿼리, 키, 값의 임베딩 관계를 학습
- N개의 인코더 레이어를 반복적으로 적용한 후, 마지막 레이어에서 분류 토큰의 특징 벡터 추출

이 분류 토큰 벡터는 이미지 데이터를 잘 표현하는 특징 벡터로 간주되어, 이후 다양한 이미지 분류 및 검색 문제를 해결하는 데 사용됨

Swin Transformer

- 기존 트랜스포머 기반 모델의 문제점
 - 고정된 패치 크기로 인해 세밀한 예측을 필요로 하는 의미론적 분할과 같은 작업에 적합하지 않음
 - 셀프 어텐션이 입력 이미지 크기에 대한 2차 계산 복잡도를 가져 고해상도 이미지를 처리하는 데 어려움
 - 계층적 특징 맵을 구상해 1차 계산 복잡도를 갖는 스윈 트랜스포머 등장
- 스윈 트랜스포머 모델
 - 시프트 윈도 기술을 이용해 입력 이미지를 일정한 크기의 패치로 분할하고 이 패치들을 쌓아 윈도우 구성

ViT vs 스윈 트랜스포머

- ViT 모델의 한계

- 획일적인 패치 크기와 위치에 대한 제약 때문에 다양한 이미지 크기와 종횡비를 가진 데이터 세트에 대한 적용이 어려움
- 패치 단위로 입력을 받아 이미지 공간 정보가 완전히 보존되지 않을 수 있음
- 이미지 패치 수 증가에 따라 학습 데이터 크기 증가로 모델 학습에 필요한 계산량 증가, 컴퓨터 자원을 많이 사용

→ 한계 보완을 위해 스윈 트랜스포머는 로컬 윈도우를 활용해 물체의 크기/해상도를 계층적으로 학습

- 스윈 트랜스포머

- 로컬 윈도우를 이동시키는 시프트 윈도우 기술 이용
 - 입력 이미지를 일정한 크기의 패치로 분할하고, 특정 맵을 이동시켜 다음 계층에 사용
- 패치의 위치를 더 유연하게 다룸!
- ⇒ ViT보다 더 높은 정확도, 더 적은 매개변수 수를 가지면서도 더 넓은 범위의 이미지 크기와 종횡비를 다룸

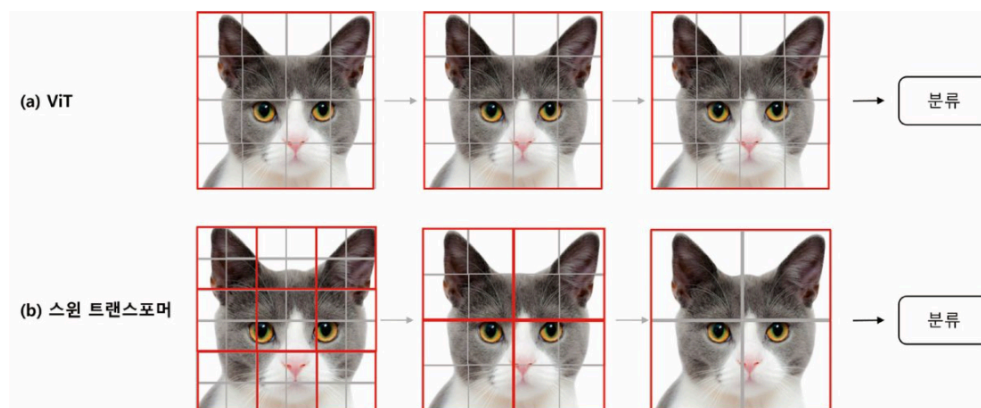


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

회색 격자는 패치, 빨간색 격자는 로컬 윈도우

ViT 네트워크: 계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기와 개수 동일
스윈 트랜스포머: 로컬 윈도우를 통해 계층마다 어텐션이 되는 패치의 크기와 개수를 계층적으로 다양하게 적용

- 계층적 접근 방식을 통해 로컬 윈도우가 각 패치의 세부 정보에 집중
 - 시프트 윈도우를 통해 전역 특징 확인
- 고해상도 이미지를 효율적으로 처리

- ViT와 스원 트랜스포머 비교

	ViT	스원 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일	다양

스원 트랜스포머 모델 구조

- 과정

1. 이미지 패치 파티션으로 구분
2. 선형 임베딩
3. 스원 트랜스포머 블록
4. 패치 병합

연산이 반복적으로 수행됨

- 이미지 구분 및 선형 임베딩

입력 이미지를 일정한 크기의 패치로 분할하고, 각 패치에 대해 선형 임베딩 수행

→ 각 패치를 고정된 차원의 벡터로 변환

- 스원 트랜스포머 블록

분할된 패치들을 기반으로 스원 트랜스포머 블록 구성

이는 어텐션 계층, 계층 정규화, 다중 퍼셉트론 등을 포함하며 패치 간 상호작용 수행

- 패치 병합

패치를 병합해 전체 이미지에 대한 분류 수행

분할된 패치들을 순차적으로 합치면서 이미지의 전체적인 정보를 추출

- 모델 구조

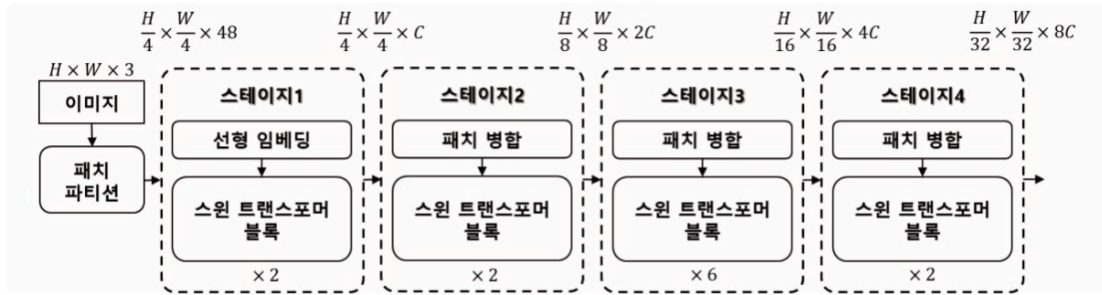


그림 10.7 스윈 트랜스포머 구조

스테이지 1에서 패치들이 선형 임베딩된 후 스윈 트랜스포머 블록에 전달
 스테이지 2,3,4에서 패치 병합 후 각 스윈 트랜스포머 블록에 전달
 패치 병합은 모델의 매개변수를 줄이기 위해 텐서를 재정렬해 저차원 임베딩을 수행

- 네 개의 스테이지로 구성
- 각 스테이지는 서로 다른 해상도/패치 크기를 가지며, 선형 임베딩/스윈 트랜스포머 블록/패치 병합이 반복적으로 적용

패치 파티션 / 패치 병합

- 패치 파티션
 - 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식
 - 분할된 패치는 트랜스포머 계층의 입력으로 사용, 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성 높임
- 패치 파티션의 구조

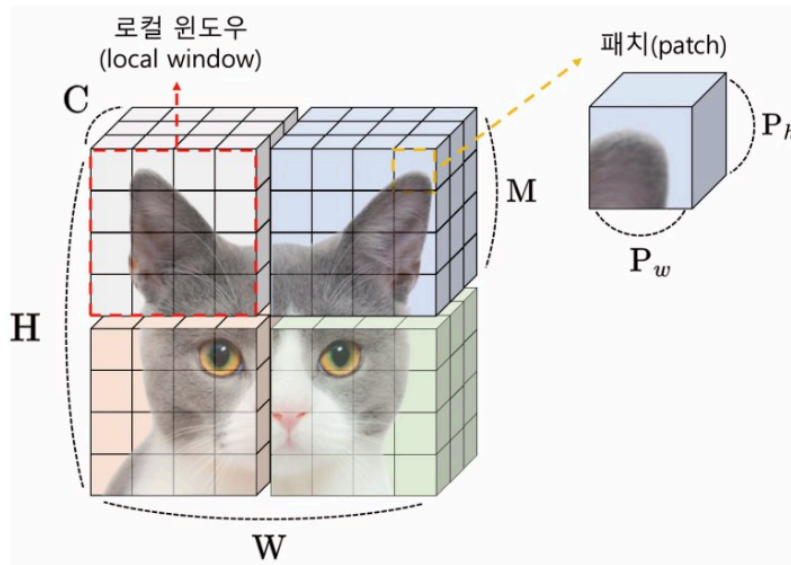


그림 10.8 패치 파티션 구조

일반적인 이미지 텐서 $[C, H, W]$ 구조에 로컬 윈도우(M) 추가

- 패치 병합
 - 인접한 패치들의 정보를 저차원으로 축소하는 과정

스윙 트랜스포머 블록

- 패치 파티션 이후 네 개의 스테이지 안에서 반복 적용
- 선형 임베딩 또는 패치 병합 이후 수행

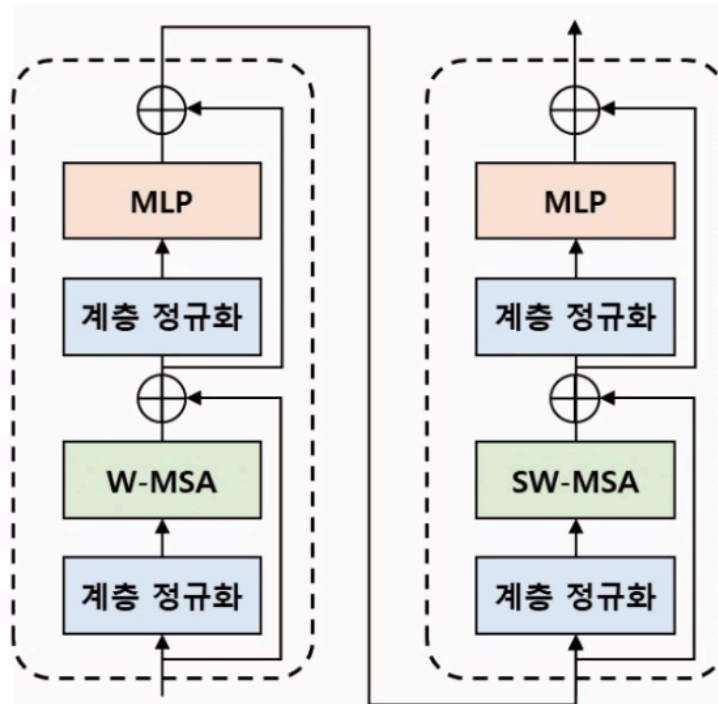


그림 10.9 스윈 트랜스포머 블록 구조

- 기존 트랜스포머 모델에서 사용되던 MSA 방식이 아닌, W-MSA와 SW-MSA로 구성, 순차적으로 수행

- W-MSA

- 로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션
- 특징 맵을 중첩되지 않게 나눈 다음 각 윈도우에서 독립적으로 셀프 어텐션 수행
- 이미지에서 일정한 크기의 윈도우 영역 설정
 - 해당 영역 내 픽셀 간 어텐션 계산
 - 이미지 내의 전체적인 어텐션 수행, 특정 위치에 대한 정보 추출
 - ⇒ 각 윈도우 내 지역적 특징 분석 가능, 전체 입력에 대한 셀프 어텐션 비용 2차 계산 복잡도에서 1차로 줄임!

- SW-MSA

- 연산량은 대폭 감소시켰지만, 이미지 패치 영역에서만 셀프 어텐션을 수행하는 W-MSA의 단점을 보완
- 윈도우 사이의 연결성 구축을 통해 윈도우 간의 관계성 정보 추출
- 윈도우를 가로 또는 세로 방향으로 이동한 후, 인접한 윈도우 간의 셀프 어텐션 계산

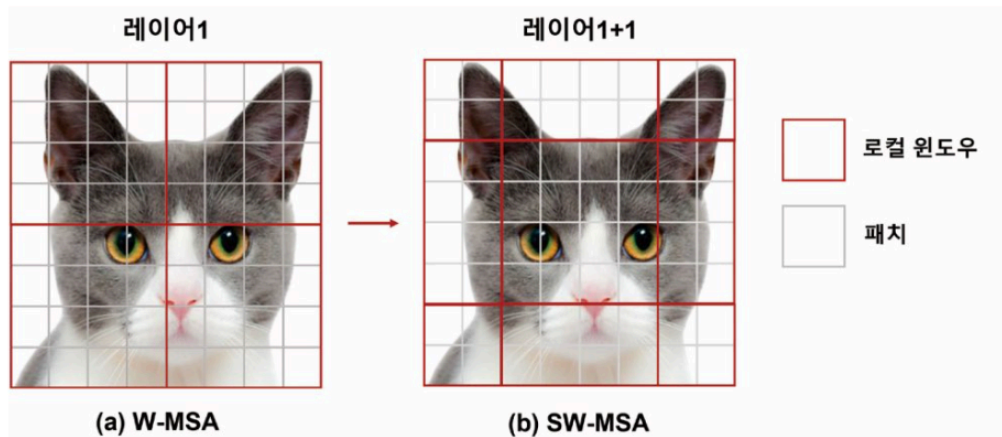


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

W-MSA: 로컬 윈도우 안에서 패치 간의 셀프 어텐션 연산 수행, 효율적인 배치 계산으로 연산량을 줄여 자연어 처리 작업에서 높은 성능을 보임

SW-MSA: 로컬 윈도우 간의 셀프 어텐션을 수행하기 위해 W-MSA에서 사용된 로컬 윈도우 개수가 많아 더 많은 셀프 어텐션 연산을 요구함, 이를 해결하기 위해 순환 시프트와 어텐션 마스크 사용

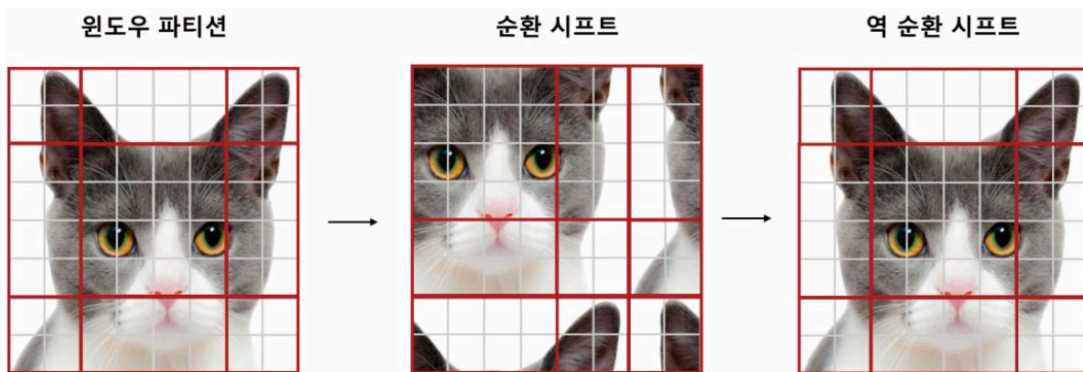


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

순환 시프트: 로컬 윈도우가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져옴

역순환 시프트: 마지막 순환 시프트 단계에서 셀프 어텐션을 수행한 다음 수행하여 로컬 윈도우 구조로 복원

CvT

- 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델
- 이미지의 물체 크기나 해상도를 계층적으로 학습하기 어려운 ViT의 문제 해결을 위해 계층형 구조를 적용
- 이미지의 지역 특징과 전역 특징을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능을 보임

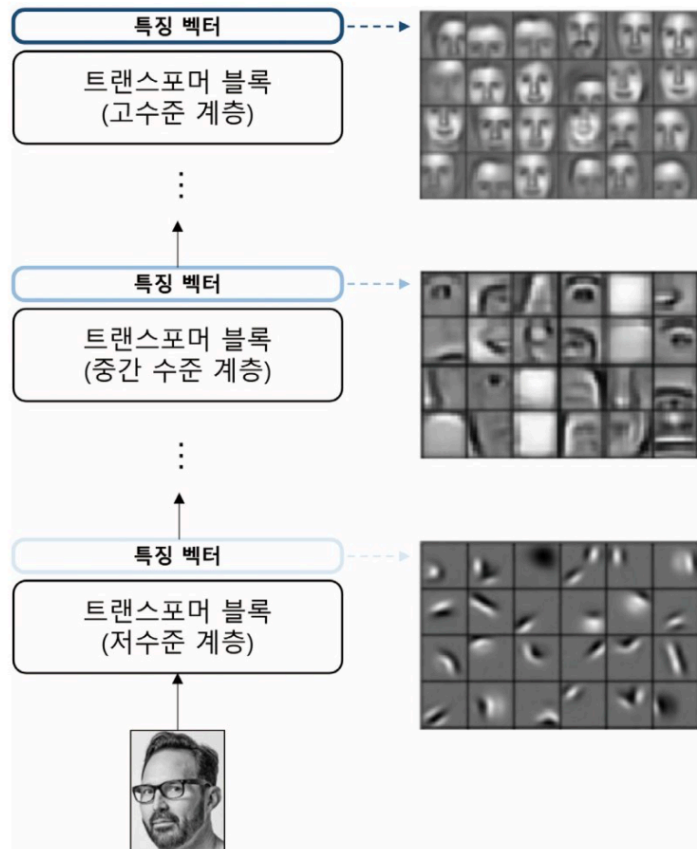


그림 10.14 CvT 모델의 계층적 특징 구조 예시

합성곱 연산을 통해 특징 맵 추출 후 유의미한 패턴을 추출하고 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용

합성곱 토큰 임베딩

- 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩 의미
- 2D 구조를 보존하면서도 셀프 어텐션을 계산하는 방식 사용
- 이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, 값 임베딩으로 변환
- 변환된 임베딩을 사용해 셀프 어텐션을 계산하고 출력 생성

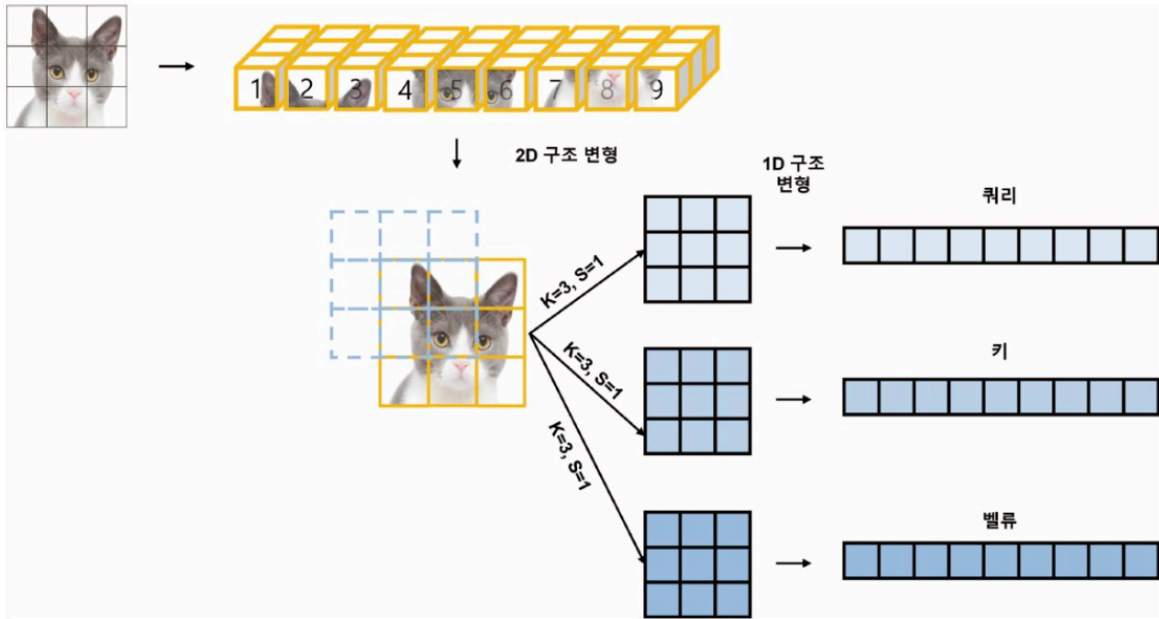


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

어텐션에 대한 합성곱 임베딩

- 2D로 재구성된 토큰맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산

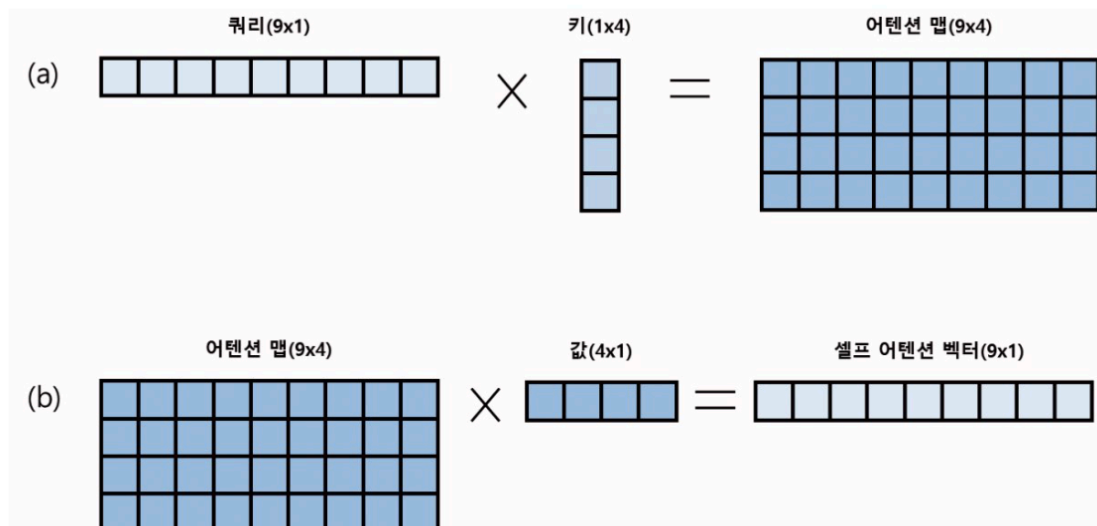


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

(a)에서 어텐션 맵 생성, 이후 (b)와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치 반영